

k近邻法

简介

主要思想

假定给定一个训练数据集，其中实例标签已定，当输入新的实例时，可以根据最近的K个训练实例的标签，预测新实例对应的标注信息。

- 分类问题：多数表决
- 回归问题：均值计算

算法流程

输入：训练集：

$$T = \{(x_1, y_1), (x_2, y_2) \cdots, (x_N, y_N)\}$$

其中， $x_i \in \mathcal{X} \subseteq \mathbf{R}^n$ ， $y \in \mathcal{Y} = \{c_1, c_2, \cdots, c_K\}$ ，实例 x ；

输出：实例 x 所属类别 y

- (1) 根据给定的距离度量，计算 x 与 T 中点的距离；
- (2) 在 T 中找到与 x 最邻近的 k 个点，涵盖这 k 个点的 x 的邻域记作 $N_k(x)$ ；
- (3) 在 $N_k(x)$ 中根据分类决策规则（如多数表决）决定 x 的类别 y 。

$$y = \arg \max_{c_j} \sum_{x_i \in N_k(x)} I(y_i = c_j), \quad i = 1, 2, \cdots, N; j = 1, 2, \cdots, K$$

三要素

距离度量

L_p 距离：

特征空间 $\mathcal{X}$ 假设为 $\mathbf{R}^n$ ， $\forall x_i, x_j \in \mathcal{X}$ ， $x_i = (x_i^{(1)}, x_i^{(2)}, \cdots, x_i^{(n)})^T$ ， $x_j = (x_j^{(1)}, x_j^{(2)}, \cdots, x_j^{(n)})^T$ ，则有

$$L_p(x_i, x_j) = \left(\sum_{l=1}^n |x_i^{(l)} - x_j^{(l)}|^p \right)^{\frac{1}{p}}, \quad p \geq 1$$

- 欧氏距离（Euclidean distance）

$$L_2(x_i, x_j) = \left(\sum_{l=1}^n |x_i^{(l)} - x_j^{(l)}|^2 \right)^{\frac{1}{2}}$$

具体有：

- 欧式距离($p = 2$)
- 曼哈顿距离($p = 1$)
- 切比雪夫距离($p = \infty$)

k-值的选择

- 较小的k值，学习的近似误差减小，但估计误差增大，敏感性增强，而且模型复杂，容易过拟合。
- 较大的k值，减少学习的估计误差，但近似误差增大，而且模型简单。

分类决策规则

多数表决规则：由输入实例的 k 个邻近的训练实例中的多数类决定输入实例的类。

- 分类函数

$$f: \mathbf{R}^n \rightarrow \{c_1, c_2, \dots, c_K\}$$

- 0-1 损失函数

$$L(Y, f(X)) = \begin{cases} 1, & Y \neq f(X) \\ 0, & Y = f(X) \end{cases}$$

- 误分类概率

$$P(Y \neq f(X)) = 1 - P(Y = f(X))$$

kd树

定义

kd树是一种对k维空间中的实例点进行存储以便对其进行快速检索的树形数据结构。

- 本质：二叉树，表示对k维空间的一个划分
- 构造过程：不断地用垂直于坐标轴的超平面将k维空间切分，形成k维超矩形区域
- kd树的每一个节点对应于一个k维超矩形区域

构造kd树

具体算法：

输入：k 维空间数据集：

$$T = \{x_1, x_2, \dots, x_N\}$$

其中， $x_i = (x_i^{(1)}, x_i^{(2)}, \dots, x_i^{(k)})^T$

输出：kd 树

(1) 开始：构造根节点。

- 选取 $x^{(1)}$ 为坐标轴，以训练集中的所有数据 $x^{(1)}$ 坐标中的中位数作为切分点，将超矩形区域切割成两个子区域，将该切分点作为根结点。
- 由根结点生出深度为 1 的左右子结点，左节点对应坐标小于切分点，右节点对应坐标大于切分点。

(2) 重复

- 对深度为 j 的结点，选择 $x^{(l)}$ 为切分坐标轴， $l = j \pmod k + 1$ ，以该结点区域中所有实例 $x^{(l)}$ 坐标的中位数作为切分点，将区域分为两个子区域。
- 生成深度为 $j + 1$ 的左、右子结点。左节点对应坐标小于切分点，右节点对应坐标大于切分点。

(3) 直到两个子区域没有实例时停止。

搜索kd树

具体算法：

输入：已构造的 kd 树，目标点 x

输出： x 的最近邻

- 寻找“当前最近点” I
 - 从根结点出发，递归访问 kd 树，找出包含 x 的叶结点；
 - 以此叶结点为“当前最近点”；
- 回溯
 - 若该结点比“当前最近点”距离目标点更近，更新“当前最近点”；
 - 当前最近点一定存在于该结点一个子结点对应的区域，检查子结点的父结点的另一子结点对应的区域是否有更近的点。
- 当回退到根结点时，搜索结束，最后的“当前最近点”即为 x 的最近邻点。